



LABORATORIO DI INTELLIGENZA ARTIFICIALE APPLICATA

A.K.A. **AI APPLICATIONS
IN PRACTICE**

3 - Python Functions

Prof. Tiberio Uricchio
< tiberio.uricchio@unipi.it >

FILES

Files

Python can be used to read and write text files.

To create or read a file, this has to be opened first (and then closed when you are done!).

A file can be opened with the **open()** function:

```
open(file_name, mode)
```

3

File creation

After opening a file, you can write in it using **write()**

```
>>> f = open('my_file.txt', 'w')
>>> for i in range(5):
    f.write('line number {}\n'.format(i))
```

Remember to close it when you are done, using the function **close()**

```
>>> f.close()
```

4

Reading a file

The content of a file can be read using the function **read()**

```
>>> f = open('my_file.txt', 'r')
>>> content = f.read()
>>> print(content)
line number 0
line number 1
line number 2
line number 3
line number 4

>>> f.close()
```

5

Reading a file

The content of a file can be read line by line using the function **readlines()**

```
>>> f = open('my_file.txt', 'r')
>>> content = f.readlines()
>>> for line in content:
    print(line)

line number 0

line number 1

line number 2

line number 3

line number 4

>>> f.close()
```

6

Append to a file

You can add new content to a file with the function **write()** but opening the file with mode **'a'**.

```
>>> f = open('my_file.txt', 'a')
>>>
>>> content = f.readlines()
>>> for line in content:
>>>     print(line)
line number 0
line number 1
line number 2
line number 3
line number 4
appendend line number 0
appendend line number 1
appendend line number 2

>>> f = open('my_file.txt', 'a')
>>> for i in range(3):
>>>     f.write('appendend line number {}'.format(i))
```

7

with ... as...

To open a file without the need to manually close it afterwards, you can use **with ... as...**

File objects have a special function **__exit__()** to be executed when exiting the *with* block which automatically closes the file.

```
>>> with open('my_file.txt') as f:
>>>     content = f.readlines()
>>>     for line in content:
>>>         print(line)
line number 0
line number 1
line number 2
line number 3
line number 4
appendend line number 0
appendend line number 1
appendend line number 2
```

8

open()

open(file_name, mode)

Mode	Description
'r'	This is the default mode. It Opens file for reading.
'w'	This Mode Opens file for writing. If file does not exist, it creates a new file. If file exists it truncates the file.
'x'	Creates a new file. If file already exists, the operation fails.
'a'	Open file in append mode. If file does not exist, it creates a new file.
't'	This is the default mode. It opens in text mode.
'b'	This opens in binary mode.
'+'	This will open a file for reading and writing (updating)

FUNCTIONS

Functions

A **function** (or *method*) is sequence of instructions to perform a specific task.

Several advantages:

- Reduced code duplication
- Better clarity
- Complex problems are decomposed into simpler parts
- Reuse of the code
- Information hiding

11

Functions

Two types of functions

- Built-in functions
- User defined functions

Functions are treated as objects: they can be assigned to variables, stored into a collection (such as a list) or passed as arguments to another function.

12

Built-in functions

Built-in Functions				
abs()	delattr()	hash()	memoryview()	set()
all()	dict()	help()	min()	setattr()
any()	dir()	hex()	next()	slice()
ascii()	divmod()	id()	object()	sorted()
bin()	enumerate()	input()	oct()	staticmethod()
bool()	eval()	int()	open()	str()
breakpoint()	exec()	isinstance()	ord()	sum()
bytearray()	filter()	issubclass()	pow()	super()
bytes()	float()	iter()	print()	tuple()
callable()	format()	len()	property()	type()
chr()	frozenset()	list()	range()	vars()
classmethod()	getattr()	locals()	repr()	zip()
compile()	globals()	map()	reversed()	__import__()
complex()	hasattr()	max()	round()	

13

Built-in functions

Built-in Functions				
abs()	delattr()	hash()	memoryview()	set()
all()	dict()	help()	min()	setattr()
any()	dir()	hex()	next()	slice()
ascii()	divmod()	id()	object()	sorted()
bin()	enumerate()	input()	oct()	staticmethod()
bool()	eval()	int()	open()	str()
breakpoint()	exec()	isinstance()	ord()	sum()
bytearray()	filter()	issubclass()	pow()	super()
bytes()	float()	iter()	print()	tuple()
callable()	format()	len()	property()	type()
chr()	frozenset()	list()	range()	vars()
classmethod()	getattr()	locals()	repr()	zip()
compile()	globals()	map()	reversed()	__import__()
complex()	hasattr()	max()	round()	

14

Functions

A function is defined with the following syntax:

```
def function_name():  
    statement1  
    statement2  
    statement3
```

```
>>> def hello():  
    print('Hello World!')
```

```
>>> hello()  
Hello World!  
>>> hello()  
Hello World!  
>>> for i in range(5):  
    hello()
```

```
Hello World!  
Hello World!  
Hello World!  
Hello World!  
Hello World!
```

15

Functions

This program is one long, complex sequence of statements.

↓

```
statement  
statement  
statement  
statement  
statement  
statement  
statement  
statement  
statement  
statement  
statement  
statement  
statement  
statement  
statement  
statement  
statement  
statement  
statement  
statement  
statement
```

In this program the task has been divided into smaller tasks, each of which is performed by a separate function.

↓

```
def function1():  
    statement  
    statement  
    statement
```

function

```
def function2():  
    statement  
    statement  
    statement
```

function

```
def function3():  
    statement  
    statement  
    statement
```

function

```
def function4():  
    statement  
    statement  
    statement
```

function

16

Functions

Function names

- Cannot correspond to python keywords (like if, for, while, def, etc)
- Cannot contain spaces
- Must start with a letter or an underscore _
- All other characters must be numbers, letters or underscores
- Are case sensitive (lowercase and uppercase are distinct)

17

Functions

A more complete example of function definition is the following, which takes two variables as inputs and returns their summation

```
>>> def compute_sum(a, b):  
    c = a + b  
    return c
```

```
>>> s = compute_sum(2, 3)  
>>> print(s)  
5
```

18

Functions

A more complete example of function definition is the following, which takes two variables as inputs and returns their summation

Keyword to declare
function

```
>>> def compute_sum(a, b):  
    c = a + b  
    return c  
  
>>> s = compute_sum(2,3)  
>>> print(s)  
5
```

19

Functions

A more complete example of function definition is the following, which takes two variables as inputs and returns their summation

Function name

```
>>> def compute_sum(a, b):  
    c = a + b  
    return c  
  
>>> s = compute_sum(2,3)  
>>> print(s)  
5
```

20

Functions

Function arguments

A more complete example of function definition is the following, which takes two variables as inputs and returns their summation

```
>>> def compute_sum(a, b):  
    c = a + b  
    return c  
  
>>> s = compute_sum(2,3)  
>>> print(s)  
5
```

21

Functions

A more complete example of function definition is the following, which takes two variables as inputs and returns their summation

```
>>> def compute_sum(a, b):  
    c = a + b  
    return c  
  
>>> s = compute_sum(2,3)  
>>> print(s)  
5
```

code block. Starts
with : and is indented

22

Functions

A more complete example of function definition is the following, which takes two variables as inputs and returns their summation

```
>>> def compute_sum(a, b):  
    c = a + b  
    return c
```

```
>>> s = compute_sum(2, 3)  
>>> print(s)  
5
```

Return statement.
Terminates the function
and returns the result to
the caller

23

Functions

A more complete example of function definition is the following, which takes two variables as inputs and returns their summation

```
>>> def compute_sum(a, b):  
    c = a + b  
    return c
```

```
>>> s = compute_sum(2, 3)  
>>> print(s)  
5
```

Function declaration

24

Functions

A more complete example of function definition is the following, which takes two variables as inputs and returns their summation

```
>>> def compute_sum(a, b):  
    c = a + b  
    return c
```

Function call

```
>>> s = compute_sum(2,3)  
>>> print(s)  
5
```

25

Functions

The ***return*** statement exits a function.

Optionally it can pass back an expression to the caller.

If a function has no return statement, it will have the same effect as “*return None*”.

```
>>> def compute_sum(a, b):  
    c = a + b  
    return c
```

```
>>> s = compute_sum(2,3)  
>>> print(s)  
5
```

26

Variable scope

The **scope** of a variable is the part of the program where it can be accessed. Variables defined inside functions are called **local variables** and they belong to the function itself. They cannot be used outside the function.

```
>>> def compute_sum(a, b):
    c = a + b
    return c

>>> s = compute_sum(2, 3)
>>> print(s)
5
>>> print(c)
Traceback (most recent call last):
  File "<pysHELL#72>", line 1, in <module>
    print(c)
NameError: name 'c' is not defined
```

27

Variable scope

Different functions may have different variables with the same name.

The two variables are completely distinct and they do not share anything.

```
>>> def f1():
    val = 5
    return val

>>> def f2():
    val = 'cat'
    return val

>>>
>>> val = 3.14
>>> print(val)
3.14
>>> f1()
5
>>> print(val)
3.14
>>> f2()
'cat'
>>> print(val)
3.14
>>> val = f1()
>>> print(val)
5
```

28

Function arguments

You can define a function with any number of arguments.

```
>>> def func():  
    print('no params')  
  
>>> def func(a):  
    print(a)  
  
>>> def func(a, b):  
    print(a, b)
```

The variables passed as arguments can be used in the function

```
>>> def hello_there(name):  
    print('hello {}'.format(name))  
  
>>> hello_there('Bob')  
hello Bob  
>>>  
>>> hello_there('John')  
hello John
```

29

Function arguments

Reference vs value

All arguments in Python functions are **passed by reference**. This means that if you change the object referred by the variable, the change also reflects outside from the function.

```
>>> def modify_list(my_list):  
    my_list.append([1, 10, 100]);  
    print('Values inside the function: {}'.format(my_list))  
  
>>>  
>>> a_list = [1, 2, 3]  
>>>  
>>> modify_list(a_list)  
Values inside the function: [1, 2, 3, [1, 10, 100]]  
>>>  
>>> print(a_list)  
[1, 2, 3, [1, 10, 100]]
```

Function arguments

- Positional arguments
- Keyword arguments
- Default arguments
- Variable-length arguments

31

Function arguments

Positional arguments

When a function is called with some arguments, the values get assigned to the arguments according to their position.

```
>>> def hello_you_two(name1, name2):  
    print('hello {}! And hello {}!'.format(name1, name2))  
  
>>> hello_you_two('Bob', 'John')  
hello Bob! And hello John!  
>>>  
>>> hello_you_two('John', 'Bob')  
hello John! And hello Bob!
```

32

Function arguments

Positional arguments

All positional arguments must be passed to the function

```
>>> def hello_you_two(name1, name2):
    print('hello {}! And hello {}!'.format(name1, name2))

>>> hello_you_two('Bob')
Traceback (most recent call last):
  File "<pyshell#47>", line 1, in <module>
    hello_you_two('Bob')
TypeError: hello_you_two() missing 1 required positional argument: 'name2'
>>>
>>> hello_you_two('Bob', 'John', 'Alice')
Traceback (most recent call last):
  File "<pyshell#49>", line 1, in <module>
    hello_you_two('Bob', 'John', 'Alice')
TypeError: hello_you_two() takes 2 positional arguments but 3 were given
```

33

Function arguments

Keyword arguments

When you call a function, you can pass arguments out of order by using specifying their names.

```
>>> def hello_you_two(name1, name2):
    print('hello {}! And hello {}!'.format(name1, name2))

>>> hello_you_two('Bob', 'John')
hello Bob! And hello John!
>>>
>>> hello_you_two(name1='Bob', name2='John')
hello Bob! And hello John!
>>>
>>> hello_you_two(name2='John', name1='Bob')
hello Bob! And hello John!
```

Function arguments

Keyword arguments

Positional and keyword arguments cannot be mixed. Positional arguments must be passed to the function before keyword arguments. The positional arguments must follow the normal ordering, while keyword arguments can be unordered.

```
>>> def sum(a, b, c):
        return a + b + c

>>> sum(1,2,3)
6
>>> sum(a=1, b=2, c=3)
6
>>> sum(1, 2, c=3)
6
>>> sum(1, c=3, b=2)
6
>>>
>>> sum(c=3, 1, 2)
SyntaxError: positional argument follows keyword argument
```

35

Function arguments

Default arguments

Arguments can be defined with default values. By doing so, the function will automatically assign the default value to the argument when it is not provided by the caller.

```
>>> def hello_you_two(name1='Bob', name2='John'):
        print('hello {}! And hello {}!'.format(name1, name2))

>>> hello_you_two('Bob', 'John')
hello Bob! And hello John!
>>>
>>> hello_you_two()
hello Bob! And hello John!
>>>
>>> hello_you_two('Alice')
hello Alice! And hello John!
>>>
>>> hello_you_two(name2='Alice')
hello Bob! And hello Alice!
```

Function arguments

Variable-length arguments

A function can accept a variable number of arguments, potentially unknown when the function itself is defined.

Variable-length arguments are passed after positional arguments and are packed into a tuple. The tuple containing the variable-length arguments can be defined with a preceding *asterisk* `*`.

37

Function arguments

Variable-length arguments

```
>>> def sum(*args):  
    tot = 0  
    for n in args:  
        tot = tot + n  
    return tot  
  
>>> sum(1, 2)  
3  
>>> sum(1, 2, 3, 4, 5)  
15  
>>>
```

38

Function arguments

Variable-length arguments

Variable-length arguments must come after positional and keyword arguments.

Typically variable-length arguments are denoted by ***args**

39

(More about unpacking...)

We have already seen something similar to what the *asterisk* is doing for variable-length arguments: *tuple unpacking*

The asterisk indicates that the tuple has to be unpacked into individual elements.

It can also be used to specify a variable-length object when unpacking

```
>>> a, b, *c = my_tuple
>>> print(a)
1
>>> print(b)
2
>>> print(c)
[3, 4, 5, 6]
```

40

(More about unpacking...)

It can also be used in different positions to unpack objects

```
>>> my_tuple = (1, 2, 3, 4, 5, 6)
>>> *a, b, c = my_tuple
>>> print(a)
[1, 2, 3, 4]
>>> print(b)
5
>>> print(c)
6
```

41

(More about unpacking...)

It can also be used in different positions to unpack objects

```
>>> my_tuple = (1, 2, 3, 4, 5, 6)
>>> a, *b, c = my_tuple
>>> print(a)
1
>>> print(b)
[2, 3, 4, 5]
>>> print(c)
6
```

42

Function arguments

Variable-length arguments

If you want to pass named arguments to the function you can use ****kwargs**, i.e. a variable-length argument where each variable has a name (a **KeyWord**).

Whereas ***args** was a tuple, ****kwargs** is a dictionary.

In practice if you want to pass a variable-length number of argument as a tuple you need to specify it with a single *asterisk* ***** and if you want to pass it as a dictionary you need to specify it with double *asterisks* ******

43

Function arguments

Variable-length arguments

```
>>> def check_args(**kwargs):
    for k, v in kwargs.items():
        print('key: {}, value: {}'.format(k, v))

>>>
>>> check_args(arg1='a', arg2='b', arg3='c')
key: arg1, value: a
key: arg2, value: b
key: arg3, value: c
```

44

Function arguments

Variable-length arguments

```
>>> def check_args(**kwargs):  
    for k, v in kwargs.items():  
        print('key: {}, value: {}'.format(k, v))  
  
>>> dict_args = {'arg1': 'a', 'arg2': 'b', 'arg3': 'c'}  
>>>  
>>> check_args(**dict_args)  
key: arg1, value: a  
key: arg2, value: b  
key: arg3, value: c
```

45

Function arguments

Variable-length arguments

When using both formal arguments and variable-length arguments, the order must be the following:

*some_func(fargs, *args, **kwargs)*

46

Recursive Functions

A function can even call itself. This is known as **recursion**.

A recursive function is first called from outside, then:

- If some condition is met, the function returns a result to the caller
- Otherwise the function calls itself again, after some processing

47

Recursive Functions

```
1 def countdown(n):
2     print(n)
3     if n == 0:
4         return
5     else:
6         countdown(n-1)
7
8     countdown(10)|
```

Shell ×

```
=====
>>> %Run countdown.py

10
9
8
7
6
5
4
3
2
1
0

>>>
```

48

Docstrings (PEP 257)

Docstrings are string literals that appear right after the definition of a function, method, class, or module. They are used to document code.

They are enclosed by triple quotes `"""` and `"""` and can be accessed via the attribute `__doc__`

```
def calculate_area(radius):  
    """  
    Calculate the area of a circle.  
  
    Parameters:  
    radius (float): The radius of the circle  
  
    Returns:  
    float: The area of the circle  
    """  
    return 3.14159 * (radius ** 2)  
  
# Access docstring  
print(calculate_area.__doc__)  
help(calculate_area)
```

49

Anonymous Functions

Anonymous functions (also called **lambda functions**) are functions defined without a name.

While regular functions are defined through **def**, anonymous functions are defined using the keyword **lambda** with the following syntax:

lambda arguments: expression

```
>>> times_two = lambda x: x * 2  
>>> print(times_two(4))  
8
```

50

Anonymous Functions

Lambda functions are useful when a short operation is required for a short period of time.

It can take any number of arguments but can only have one expression

```
>>> multiply = lambda x, y: x*y
>>> multiply(3,5)
15
```

51

Anonymous Functions

The power of lambda functions is better expressed when used inside other functions.

```
>>> def times_n(n):
    return lambda a : a * n

>>> times_3 = times_n(3)
>>> print(times_3)
<function times_n.<locals>.<lambda> at 0x000002B0970801E0>
>>> times_3(6)
18
>>> times_5 = times_n(5)
>>> times_5(2)
10
```

52

map()

Lambda functions are often used with the **map()** function.

Map takes as arguments a function and a list of arguments. It applies the function to all arguments in the list.

```
>>> my_list = [1, 5, 4, 6, 8, 11, 3, 12]
>>> new_list = list(map(lambda x: x * 2 , my_list))
>>> print(new_list)
[2, 10, 8, 12, 16, 22, 6, 24]
```

53

filter()

Lambda functions are often used with the **filter()** function.

Filter takes as arguments a function and a list of arguments. It applies the function to all arguments in the list.

A new list is returned containing only the items for which the function returned **True**.

```
>>> my_list = [1, 5, 4, 6, 8, 11, 3, 12]
>>> new_list = list(filter(lambda x: (x%2 == 0) , my_list))
>>> print(new_list)
[4, 6, 8, 12]
```

54

any()

The **any()** function is a built-in Python function that returns **True** if at least one element in an iterable is **True**. If the iterable is empty, it returns **False**.

```
# Returns True because at least one element (True) evaluates to True
print(any([False, True, False])) # Output: True

# Returns False because all elements evaluate to False
print(any([False, False, False])) # Output: False

# Returns False for an empty iterable
print(any([])) # Output: False
```

55

any()

Some practical examples

```
# Check if any number in a list is greater than 10
numbers = [2, 5, 8, 12, 7]
print(any(num > 10 for num in numbers)) # Output: True

# Check if any string in a list contains the letter 'a'
words = ["python", "code", "functions"]
print(any('a' in word for word in words)) # Output: False

# Check if any element in a list is even
print(any(x % 2 == 0 for x in [1, 3, 5, 7, 9])) # Output: False
```

56

Exercises

- Write a function to find the max of some integers (unknown number of integers in the args). The max must be above a user-defined threshold. If not, print a message.
- Write a function to check whether at least one number among some given as input is above a user-defined threshold using filter().
- Write a function to obtain a list from the numerical values of a dictionary converted into strings.

MODULES

Modules

Python modules allow you to organize your code.

A module is a file with python code which can contain the definitions of functions, classes and variables.

Modules can be imported and used in other scripts.

59

Modules

Everything that is defined inside a module can be used by importing the module.

A module can be imported with

```
import module_name
```

A module can be imported if it can be found in Python's search path.

A search path is a list of directories in which the Python interpreter searches before importing a module (more on this later).

It can be modified through the environment variable PYTHONPATH and it also includes the current directory.

60

Modules

To use functions defined in a module you just need to import it and call the functions with

```
import module_name  
  
module_name.function_name()
```

61

Modules

Objects inside a module_name can be listed with dir()

```
import module_name  
  
dir(module_name)
```

62

Modules

There are several ways to import a module...

```
import module_name  
  
module_name.function_name()
```

63

Modules

There are several ways to import a module...

```
from module_name import function_name  
  
function_name()
```

64

Modules

There are several ways to import a module...

```
from module_name import *  
  
function_name()
```

65

Modules

There are several ways to import a module...

```
import module_name as m  
  
m.function_name()
```

66

Modules

If you change a module and you want to import it again in your code, you can use

```
reload(module_name)
```

67

Packages

A package is a hierarchical file directory structure defining modules and subpackages...

...it means you can put files in folders

68

Packages

A package is a hierarchical file directory structure defining modules and subpackages...

...it means you can put files in folders

You can then import a module inside a package with

```
import package_name.module_name  
  
package_name.module_name.function_name()
```

69

Packages

Using “as” will make your life easier

```
import package_name.module_name as m  
  
m.function_name()
```

70

__init__.py

In a module we may want to export only some of the functions defined or need an initialisation code for the package.

`__init__.py` is a special file used to mark a directory as a Python package.

```
my_package/  
|-- __init__.py  
|-- module1.py  
|-- module2.py
```

71

__init__.py

You can leave it empty just to mark a directory as a package.

You can set up package-level variables.

You can set `__all__` to control what gets imported **with `import *`**

```
# Inside my_package/__init__.py  
print("Initializing my_package")  
__all__ = ["module1"]  
  
# from a script  
import my_package # Prints "Initializing my_package"  
from my_package import * # Only imports module1
```

72

Naming functions

Functions are always public but the indication of one underscore at the beginning suggests internal use only.

```
def _internal_method(self): # Suggests internal use
    print("This is a non-public method")
```

73

Private methods

Let's say we have two modules inside a package **mypackage**

—mypackage/module1.py

```
def public_function():
    return "This is a public function"

def _private_function():
    return "This is a private function"
```

—mypackage/module2.py

```
def hidden_function():
    return "This function should not be called from outside of the package"
```

74

Private methods

In **mypackage/__init__.py** we can choose what to make public

1 - use `__all__`

```
from .module1 import *  
  
__all__ = ["public_function"]
```

2 - import only what we want to expose

```
from .module1 import public_function
```

75

Libraries

A Python library is made of reusable code you may want to include in several programs.

A library is basically a collection of modules.

Some libraries are already installed with Python, you may want to install others...

76

The standard library

The Python Standard Library is a library that comes directly with Python and is divided into several useful packages.

Let's see some examples

77

The standard library

```
import os
os.getcwd()          # Return the current working directory

os.chdir('/server/accesslogs')  # Change working directory
```

78

The standard library

```
import shutil
shutil.copyfile('data.db', 'archive.db')

shutil.move('/build/executables', 'installdir')
```

79

The standard library

```
>>> import glob
>>> glob.glob('*.py')
['myfile.py', 'script.py', 'exercise.py']
```

80

The standard library

```
>>> import math
>>> math.cos(math.pi / 4)
0.70710678118654757
>>> math.log(1024, 2)
10.0
```

81

The standard library

```
>>> import random
>>> random.choice(['apple', 'pear', 'banana'])
'apple'
>>> random.sample(range(100), 10)    # sampling without replacement
[30, 83, 16, 4, 8, 81, 41, 50, 18, 33]
>>> random.random()                 # random float
0.17970987693706186
>>> random.randrange(6)             # random integer chosen from range(6)
4
```

82

The standard library

```
>>> import statistics
>>> data = [2.75, 1.75, 1.25, 0.25, 0.5, 1.25, 3.5]
>>> statistics.mean(data)
1.6071428571428572
>>> statistics.median(data)
1.25
>>> statistics.variance(data)
1.3720238095238095
```

83

The standard library

```
>> import time
>> t = time.time()
>> my_slow_function() # let's see how much time this takes
>> print('{} seconds'.format(time.time() - t))
5.746150493621826 seconds
```

84

PyPI

In addition to the standard library, several additional libraries can be installed through the Python Package Index (PyPI): <https://pypi.org/>

A library can be installed with **pip**

```
python -m pip install package_name
```

or simply

```
pip install package_name
```